Qoyma Twin: подробное техническое задание

Версия 1.0, 06.11.2026. Материал для начала разработки двумя участниками за 48 календарных часов. Продукт ещё не реализован; проверка донорского репозитория и подготовка данных описаны отдельно. При конфликте примеров и текста приоритет имеют правила этого ТЗ и файл contracts/README.md.

0. Результат проверки идеи перед разработкой

На подготовленных данных выполнены отдельные исследовательские расчёты маршрутов и труда, без очередей и полноценного приложения. При сильном исходном ABC-размещении и честной цене переноса ML и mean28 предлагают **не менять размещение во все20 обычных тестовых дней**. Четыре пятидневных окна также дали no-op. Это ограничение конкретного двадцатитоварного benchmark, не доказательство бесполезности slotting вообще.

Поэтому MVP имеет два явно разделённых сценария:

- «Обычный исторический спрос»: система показывает прогноз и может рекомендовать ничего не менять. Результат защищает от лишней работы.
- «Запланированная акция»: заранее известное условие 5× спроса для SKU22382 на 03.10.2011, остальные SKU без модификации. Это синтетический what-if поверх базового прогноза, не предсказанная нейросетью акция и не реальная история UCI. Оба метода предлагают обмен22382↔85123A; ожидаемая суррогатная выгода уже покрывает перенос125.67c. Фактический эффект по маршрутам/очередям для этого сценария пока не посчитан и должен быть проверен при реализации.

В сцене и отчёте эти режимы не смешиваются. Нельзя обещать, что ML уже дал экономию: он не превзошёл mean28 по дневной ошибке, а в синтетической акции оба метода выбрали один обмен. Недельный ML-прогноз точнее baseline в небольшом дополнительном probe, но тоже не дал полезных переносов. Точные числа и границы - в 06_Data_ML_and_Feasibility.md и data.

1. Что строим

Веб-приложение, в котором руководитель склада видит двадцать товаров на трёхмерных стеллажах, получает прогноз спроса на следующую дату, выбирает ограниченную перестановку и проверяет её на цифровой модели. Два одинаковых склада получают одинаковые заказы: исходное размещение и предложенное размещение. Пользователь видит маршруты двух комплектовщиков, загрузку упаковочного поста и результат с учётом труда на перестановку.

Продуктовая формулировка: «Какие две пары товаров стоит поменять местами перед следующей сменой, чтобы ожидаемая экономия отбора оправдала работу по переносу?» Модель вправе ответить «перестановка не окупается». Это полноценный результат.

Заказчик первой гипотезы: небольшой склад интернет-магазина или операционный руководитель участка комплектации. Нужные входы будущего пилота: история строк заказов, действующие адреса SKU, план проходов, замер скорости ходьбы и операций. Подключение к WMS и датчикам отложено.

2. Обязательный объём и граница проекта

P0, без чего демонстрация не считается готовой:

1. Один склад 20 × 18 м, четыре прохода, двадцать адресуемых ячеек, двадцать SKU, свободный буфер, два комплектовщика и один упаковочный пост.

2. Реальные исторические продажи UCI; подготовленный локальный ассортиментный срез без CustomerID. Источник, фильтры и даты видны в разделе «Данные и допущения».
3. Один обученный CPU-прогноз и минимум два простых прогноза для сравнения. Прогнозируется число посещений SKU в заказах за сутки.
4. Исходное ABC-размещение по прошлой частоте отбора; планы «без изменений», «до одного обмена» и «до двух обменов».
5. Граф маршрутов с непроходимыми стеллажами. Детерминированная симуляция двух комплектовщиков и очереди к одному упаковщику.
6. Парный просмотр с едиными временем и камерой, пауза, ускорение, перемотка, сброс, подсветка переносимых SKU и выбранного маршрута.
7. Итоговые метрики из событий расчётного ядра, включая стоимость перестановки. Значения не задаются декоративными константами.
8. Два подписанных локальных режима без интернета: исторический контроль и запланированная акция; краткий отчёт по всем заранее выбранным тестовым датам.

P1, только после P0: ввод интенсивности потока, сценарий «ускоренная упаковка», экспорт отчёта, простое объяснение результата через API LLM, редактирование параметров в пределах допустимых значений.

За пределами двух дней: физическая симуляция тел, предотвращение столкновений людей, роботы/ROS/Isaac Sim, обучение RL, распознавание товара камерой, пополнение запасов, реальная инвентаризация, универсальный редактор CAD, авторизация, многопользовательская работа, коммерческие интеграции, полноценное управление складом. Отсутствие этих функций должно быть видно в описании модели.

3. Выбор стека

Основной путь: Vue 3 + TypeScript + Vite + plain Three.js для интерфейса; Python 3.11/3.12 + pandas + openpyxl + scikit-learn + SimPy + FastAPI/Pydantic для расчётов. Это сохраняет стек проверенного донора Xxcool/smart-warehouse и позволяет использовать готовую загрузку GLB, камеру и выбор объектов.

Не переходить на React Three Fiber ради одного пакета. Если команда фактически знает только React и ещё не начала адаптацию, допустим отдельный React/Vite фронтенд, но решение принимается в первый час и затем замораживается. Нельзя одновременно поддерживать две реализации.

Данные, расчёты и обучение работают на CPU. LLM необязателен и не участвует в определении путей, длительностей, перестановок или KPI. Все ключи находятся только в переменных окружения backend; ключ не попадает в Vite bundle.

Для MVP достаточно файлов JSON/CSV и словаря результатов в памяти процесса. PostgreSQL, Redis, брокер сообщений и Docker не входят в критический путь. Зависимости фиксируются lockfile/requirements после первого успешного прогона.

4. Пользовательский сценарий и интерфейс

Один рабочий экран на ноутбук 1366 × 768 и выше:

- Сверху: дата прогноза, источник сценария, число заказов/строк, кнопки «Рассчитать планы» и «Сравнить».
- Основная область: два вида одного склада, слева «Текущее ABC», справа «План: до 2 обменов». Над каждой сценой число выполненных заказов, путь и состояние упаковки.
- Левая панель или вкладка: спрос по SKU, прогноз и факт после раскрытия тестового дня; статус модели и простые прогнозы.
- Правая панель: три плана, перемещаемые SKU, три физических переноса на обмен, ожидаемые затраты и причина выбора.

- Снизу: единая шкала времени, Play/Pause, 1×/10×/60×, сброс, режим «маршруты» и «нагрузка проходов».
- После окончания: таблица результата, отдельно прогнозные оценки и результат на отложенных заказах; кнопка «Данные и допущения».

Не выводить десятки декоративных графиков. Главное действие: выбрать план и увидеть последствия. Цвета: исходный вариант янтарный, предложенный бирюзовый; подписи обязательны, чтобы сравнение читалось без различения цветов. Движение комплектовщика не должно зависеть от частоты кадров.

Состояния интерфейса:

Состояние	Поведение
Загрузка модели/данных	Показать этап и прогресс GLB, кнопки расчёта недоступны
Готово к расчёту	Отобразить выбранную дату, параметры и состояние baseline
Идёт расчёт	Показать текущий этап; повторная кнопка не создаёт второй запрос
Планы готовы	Можно просмотреть переносы и запустить сравнение
Нет полезного плана	«Экономия не покрывает переносы», доступен план без изменений
Проигрывание/пауза/конец	Управление общей шкалой, KPI соответствуют выбранному времени
Ошибка API	Сообщение с повтором; ранее готовые результаты сохраняются
Нет WebGL/GLB не загрузился	2D-план проходов и реальные расчётные результаты либо явное сообщение; не пустой экран
Нет заказов на дату	Нулевой поток без деления на ноль; нельзя обещать выигрыш
Несовместимый layout_version	Заблокировать показ маршрутов и сообщить причину

Пользователь меняет параметры только до запуска нового расчёта. Старый результат помечается устаревшим, но не смешивается с новыми параметрами. Достаточно обычных POST-запросов с расчётом до нескольких секунд; фоновые очереди и WebSocket не нужны.

5. Данные и правила подготовки

Источник: UCI Online Retail II, британский интернет-магазин подарочных товаров за 2009-2011 годы, CC BY 4.0. Это история транзакций, не журнал перемещения работников. Геометрия, доступность товаров, рабочая смена и операционные длительности задаются нами.

Порядок обработки:

1. Прочитать оба листа XLSX и привести отличающиеся заголовки к invoice_id, sku_id, description, quantity, invoice_at, unit_price, country.
2. Идентификаторы хранить строками. Удалить строки без invoice/sku/date, отмены с префиксом C, quantity ≤ 0 и unit_price ≤ 0. В подготовленном срезе merchandise определяется явным шаблоном StockCode: пять цифр и необязательная одна буква. Это воспроизводимое допущение, исключающее и некоторые нестандартные коды; количество исключений хранится в manifest. Не отбрасывать строку только из-за отсутствующего CustomerID.
3. Зафиксировать политику точных дублей: одинаковые исходные строки считать дублями выгрузки и убрать один раз; число удалённых строк сохранить. Это допущение, поскольку идентификаторов исходных строк нет.
4. Объединить повторные (invoice_id, sku_id): quantity суммируется, visit = 1, время берётся самое раннее. Для накладных с несколькими timestamps также использовать самое раннее время для поступления и сохранить число таких случаев. В подготовленных данных 81 накладная имеет несколько timestamps; это не скрывать.

5. До выбора SKU разделить время. Фиксированная граница основного holdout: 2011-10-03, 00:00 по локальному времени исходного набора без придумывания timezone. Данные раньше границы доступны модели и отбору ассортимента.
6. Выбрать 20 стабильных SKU только по истории до границы. Не выбирать товары и даты по величине будущего выигрыша. Правило выбора, включая минимальную активность, записать в manifest; после подготовки не менять.
7. Сохранить все строки этих SKU внутри накладных; пустые после проекции накладные убрать. Назвать это ассортиментным срезом. Сохранить долю исходных накладных, строк и количества, покрытую срезом.
8. Построить календарную сетку date × sku с нулевыми днями. Отсутствие продажи и отсутствие исходного дня различать в manifest. Рабочие дни не выводить из будущей активности магазина.
9. Сохранить daily_visits, daily_units, каталог и обезличенные тестовые корзины. CustomerID не нужен и не экспортируется.

Дата InvoiceDate служит приближением времени поступления задачи. Это не подтверждённый timestamp заказа из WMS. В историческом replay используем относительные интервалы внутри выбранного дня; если выносим их в смену 09:00-17:00, сохраняем порядок и явно указываем коэффициент преобразования. Для стресс-сценария разрешено одинаковое сжатие интервалов в обоих вариантах с пометкой «синтетическая интенсивность».

Правило будущей материализации акции для P0: взять срез первой тестовой даты, сохранить исходные заказы; для каждой исходной строки SKU22382 создать ещё четыре однострочных заказа с тем же quantity и arrival_s, новым стабильным order_id. Это увеличит фактические visits именно этого SKU в5раз, не умножая остальные позиции корзин. Сохранить единый scenario JSON до запуска обоих вариантов. Все добавленные заказы синтетические; их нельзя выдавать за реальные UCI. Модификатор5× применяется к прогнозу до планирования, факт такого же сценария используется только для последующей оценки. Множитель не подбирается по максимальному выигрышу.

Первый день может начинаться с очереди уже поступивших заказов, если ранние timestamps попадают до начала заданной смены; правило явно фиксируется. Предпочтительный простой режим: начало = первый timestamp выбранного дня, relative_arrival = timestamp - начало. Никакой случайной перестановки заказов.

6. Прогноз спроса и оценка AI

Целевая переменная visits[sku, date] = число разных накладных, содержащих SKU. Количество единиц применяется к длительности операций отдельно. Один заказ с 100 единицами SKU создаёт одно посещение этого SKU.

Простые модели: seasonal naive y[t-7], среднее прошлых 28 календарных дней. Основная модель: общий для 20 SKU HistGradientBoostingRegressor с Poisson loss и OneHotEncoder SKU. В выполненном probe: max_iter=150, max_leaf_nodes=15, learning_rate=0.06, l2_regularization=1, random_state=42; параметры заданы заранее без подбора на test. Запасной вариант при несовместимости окружения: RandomForestRegressor с one-hot SKU или любой устойчивый CPU-регрессор с теми же временными границами. Дополнительный подбор допускается только на validation.

Признаки: SKU, день недели, месяц, lag 1/2/7/14/28, средние 7/28 с обязательным shift(1), при необходимости отношение краткого и длинного среднего. Все признаки строятся из доступного прошлого. Для ближайшего следующего дня будущие значения не требуются. Не добавлять факт завтрашнего объёма заказов как признак. Выполненный probe обучен на weekday-targets с минимум28 днями после первого наблюдения SKU; реальные выходные остаются в истории лагов.

Рекомендуемый протокол:

- Validation: последние 28 календарных дней перед 2011-10-03; обучение на более ранней истории, rolling-origin прогноз на один день вперёд. Подбор ограничить несколькими заранее заданными конфигурациями.

- Test: первые 20 наблюдаемых дат ПН-ПТ начиная с 2011-10-03: в подготовленном наборе это 3-28 октября 2011 года. Это исторический benchmark рабочих дней; календарные нули и реальные продажи по выходным сохраняются в признаках. На каждую дату использовать только уже наступившую историю. Можно обучить модель один раз в начале test и обновлять lag-признаки после раскрытия очередного дня.
- Предпочтительная упрощённая версия: одна модель, обученная до cutoff, без поиска гиперпараметров; те же 20 дат. Явно подписать fixed-model rolling-origin.
- Для демонстрации выбрать заранее первую тестовую дату с непустым срезом, а не день с максимальным эффектом. Все остальные даты входят в итоговую таблицу.

Метрики: MAE и WAPE = sum(abs(y - yhat)) / sum(y). При нулевом знаменателе WAPE = null с подписью «нет спроса». Не усреднять MAPE по нулевым дням. Дополнительно можно показать качество ранжирования топ-5 SKU, но оно не заменяет операционный результат.

Отрицательные предсказания обрезаются на нуле; дробные значения сохраняются в планировщике. Не округлять спрос до построения плана. Прогноз без истории для SKU -> простая модель и статус fallback.

Обязательное разделение экспериментов:

1. ABC без переносов.
2. Тот же ограниченный планировщик с seasonal naive/rolling mean.
3. Тот же планировщик с ML-прогнозом.

Так видно, что даёт ML отдельно от самой перестановки. Oracle на фактическом спросе допустим как отдельно помеченный недоступный в реальности ориентир; он не участвует в выборе демонстрационного плана. ML может проиграть простой модели, и отчёт должен сохранить этот результат.

7. Геометрия и навигация

Канонические координаты: метры, правосторонняя система, Y вверх, пол X/Z. Прямоугольник пола X 0…20, Z 0…18. Четыре центра проходов: X = 3, 7, 11, 15. Передний поперечный проход Z = 2, задний Z = 15. Пять точек отбора на каждом проходе: Z = 4, 6, 8, 10, 12.

Двадцать ячеек: S_A1_B1 … S_A4_B5. Ячейка обслуживается из точки (aisle_x, 0, bay_z); коробки находятся на стеллаже слева от прохода, примерно (aisle_x - 1.5, 0.9, bay_z). Габариты одной секции 0.9 × 2.6 × 1.5 м по X/Y/Z. Декоративные верхние полки не создают дополнительные SKU.

Упаковочный стол: mesh-центр (9, 0.85, 0.6), доступная работнику точка N_PACK = (9, 0, 1.4), связанная с N_FRONT_9 = (9, 0, 2). Буфер: N_BUFFER = (18, 0, 2), контейнер стоит в отмеченной зоне около (18, 0.3, 0.8).

Граф содержит 31 узел и 33 неориентированных ребра: четыре вертикальных цепочки по семь точек, связи между соседями на переднем и заднем поперечных проходах, дополнительный передний узел X=9, буфер X=18 и упаковочный отвод. Вес ребра равен евклидовой длине его прямого отрезка в метрах. Каждый отрезок лежит в проходе. Не строить полный Manhattan-граф между всеми ячейками: он разрешит пересекать стеллажи.

Пути вычисляются Dijkstra; при равных расстояниях порядок соседей и ID фиксированы. Допустимо заранее рассчитать матрицу кратчайших путей между 20 точками отбора, упаковкой и буфером. Геометрия и назначение SKU разделены: перестановка товара не меняет граф.

Комплектация одного заказа: старт у упаковки, greedy nearest-neighbor по оставшимся SKU через граф, tie-break sku_id, возвращение к упаковке. Одинаковое правило для всех размещений. Это эвристика маршрута, не доказанный глобальный оптимум. На полке работник останавливается, «рука/коробка» анимируется локально, но логическая позиция остаётся на доступной точке прохода.

8. Исходное размещение и ограниченный планировщик

Baseline ABC создаётся один раз на cutoff 2011-10-03: товары ранжируются по числу pick visits за предшествующие 28 календарных дней; ячейки - по расстоянию от упаковки, затем slot_id. Самый частый товар получает ближайшую ячейку. Это уже существующее размещение, одинаковое для всех 20 тестовых дат. Стоимость его первоначального создания не учитывается ни у одного метода.

Каждая тестовая дата - независимое решение относительно этой зафиксированной базы: утром предлагается ограниченный план и учитывается его полная стоимость. Накопленные изменения размещения между тестовыми днями не моделируются. Поэтому сумма результатов - benchmark независимых решений, а не точный финансовый результат месяца непрерывной эксплуатации. Daily ABC можно добавить позже отдельной сильной контрольной стратегией, но не смешивать с исходной базой.

Планы: 0, максимум 1 swap, максимум 2 swaps. Одна swap затрагивает 2 разных SKU, две непересекающиеся swap - максимум 4 SKU. Буфер пуст до и после плана. Остальные SKU остаются на местах. Ограничение на бюджет не означает обязательное использование всего бюджета.

Основная целевая оценка в человеко-секундах:

J(L) = Σ_s predicted_visits_s × 2 × graph_distance(pack, slot_L(s)) / walk_speed + relocation_labor_seconds(L0 > L).

Первое слагаемое - приближение одиночными посещениями, а не точное время многотоварных заказов. Это прозрачная дешёвая функция для генерации кандидатов. Реальные корзины затем оцениваются маршрутной симуляцией. Не подписывать J как «точно прогнозируемое время смены».

Алгоритм MVP:

1. Посчитать J для исходного размещения без переносов.
2. Перебрать все 190 пар SKU, для каждой построить допустимую swap и выполнимые переносы через буфер.
3. Выбрать пару с максимальным положительным снижением J. Равенство разрешить по отсортированным SKU-ID.
4. Для плана до двух swap повторить поиск среди ещё не перемещённых SKU; сравнивать суммарный J с исходным вариантом и текущим лучшим.
5. Вернуть планы 0/1/2 с фактическим количеством обменов, трудозатратами и ожидаемым изменением J. При отсутствии положительного эффекта вернуть исходный вариант.

Для P1 кандидатов можно дополнительно проверять на прогнозном потоке, построенном только из прошлых корзин, с явной маркировкой генератора. Фактические будущие корзины запрещено использовать для выбора плана. В P0 достаточно честно отделить surrogate-оценку планировщика от оценки на отложенном дне.

Физическое исполнение swap A↔B: A > buffer, B > A, buffer > B. Один SKU представлен стандартизированным контейнером; размеры и масса реальных товаров неизвестны. Все двадцать ячеек в MVP совместимы, ограничение вместимости/температуры/массы не моделируется. Нельзя обещать, что этот план можно сразу применять на реальном складе без проверки этих ограничений.

Стоимость одной swap: кратчайшая замкнутая пешая последовательность одного исполнителя pack > A > buffer > B > A > buffer > B > pack плюс 6 операций take/place. Разрешено проверить обе ориентации обмена и выбрать меньшую. Для двух swap обе выполняются последовательно тем же исполнителем; бюджет - 2 swap, не 2 физических переноса. Формула и последовательность должны совпадать с plan.transfers.

9. Расчётная симуляция

Время расчёта - непрерывные секунды, начало 0. Начальный состав: два свободных комплектовщика у N_PACK, пустая очередь, один свободный упаковщик; доступность выбранных SKU неограниченная. Это позволяет изолировать размещение, но исключает выводы о дефиците и обороте запасов.

Правила:

- Заказы поступают в заданные arrival_s, одинаковые в обоих прогонах. Сортировка (arrival_s, order_id).
- Один комплектовщик получает целый заказ из FIFO. При одновременной доступности первым назначается P1, затем P2. Деление заказа между людьми и batch-picking не реализуются.
- Комплектовщик следует готовому маршруту, останавливается у каждого SKU и возвращается к упаковке.
- После сдачи корзины в неограниченную очередь упаковки комплектовщик сразу свободен. Он не ждёт завершения упаковки. Это отдельное явное допущение. picked_last_at_s означает отбор последнего SKU; pick_finished_s означает завершение возврата и dropoff, то есть момент готовности корзины к упаковке. Для ожидания упаковки используется только второй timestamp.
- Один упаковщик обрабатывает FIFO; передача двух корзин в один момент разрешается по order_id. Заказ считается выполненным только после упаковки.
- Узкие проходы и блокировки между людьми не моделируются. Визуальное расхождение двух аватаров на 0.15 м - косметика, оно не меняет путь/время/KPI.
- Прогон продолжается до упаковки последнего заказа. Дополнительно считается число завершённых к концу фиксированного горизонта H; не путать это с выполнением всего потока.

Стартовые параметры для демонстрации, все являются допущениями:

Параметр	Значение	Смысл
walk_speed_mps	1.2	Одинаковая постоянная скорость обоих комплектовщиков
pick_base_s	6	Фиксированное время одного посещения SKU
pick_per_unit_s	0.15	Добавочное время на каждую единицу в строке
dropoff_s	3	Сдача готовой корзины на упаковку
pack_base_s	20	Фиксированная часть упаковки заказа
pack_per_line_s	2	Добавка за каждый SKU заказа
pack_per_unit_s	0.05	Добавка за каждую единицу
relocation_handle_s	8	Одно take или place контейнера
relocation_worker_count	1	Перестановки до смены выполняются последовательно
horizon_s	28800	Условная 8-часовая смена для completed_by_horizon

Формулы: pick_s = 6 + 0.15 × quantity; pack_s = 20 + 2 × distinct_sku_count + 0.05 × units; travel_s = distance_m / 1.2. Большие количества не обрезаются тайно. Если вводится ограничение допустимой корзины, оно заранее применяется ко всем вариантам и показывается в manifest вместе с исключённой долей.

Перестановки считаются выполненными до поступления дневного потока, поэтому не задерживают arrival_s. Их человеко-секунды вычитаются из экономии труда отдельно. Не вычитать их автоматически из календарного makespan и не складывать человеко-часы с временем ожидания клиента.

Реализация: SimPy Environment, FIFO очереди заказов и упаковки, два процесса комплектовщиков, один процесс упаковщика. Равные timestamps обрабатываются устойчиво. Допустим компактный собственный heapq event engine, если SimPy мешает, но одновременно два разных движка не поддерживаются.

10. События, воспроизведение и метрики

Единственный источник правды - результат backend. Фронтенд получает интервалы действий работников и события заказов. Он интерполирует заданные отрезки и отображает состояния. Ни одна анимация не создаёт дополнительные операции, маршруты или готовые заказы.

Для движения: отдельные WALK-интервалы с массивом waypoint и монотонными at_s; на участке между двумя точками линейная интерполяция, без CatmullRom-срезания углов. Для работы на месте: интервал pick/dropoff/idle. Вне WALK аватар остаётся в заданной позиции; нельзя интерполировать одним сплошным путём через остановку и протащить человека во время picking. Для заказов: arrival, assigned, picked_last, pick_finished, pack_started, completed. Событие несёт order_id, SKU/slot при необходимости и точное simulation time.

Все интервалы полуоткрытые [start_s, end_s). Состояние на границе определяется seq. При seek состояние полностью восстанавливается из snapshot + событий/интервалов, а не «догоняется» вызовом всех кадров с начала. После reset счётчики, trail и очередь возвращаются к нулю. После конца первого варианта его сцена остаётся в финальном состоянии, пока второй доигрывает на общей шкале.

Метрики итогового отчёта:

- total_distance_m - сумма расстояний двух комплектовщиков только при выполнении заказов; relocation_distance отдельно.
- mean_cycle_s и p95_cycle_s - completed - arrival по заказам, с явно зафиксированным методом percentile.
- mean_pick_cycle_s - pick_finished - assigned, без ожидания заказа в общей очереди.
- makespan_s - завершение последнего заказа минус 0; включает естественные интервалы поступления, поэтому может почти не измениться при снижении пути.
- completed_by_horizon - количество упакованных к H; backlog_at_horizon - поступившие к H, но не завершённые.
- mean_pack_wait_s - pack_started - pick_finished; max_pack_queue и time_weighted_mean_pack_queue - очередь без обслуживаемого заказа.
- picker_busy_s - движение + отбор + сдача по каждому человеку. packer_busy_s отдельно.
- gross_pick_labor_saved_s = baseline_picker_busy_s - candidate_picker_busy_s.
- net_labor_saved_s = gross_pick_labor_saved_s - relocation_labor_s. Отрицательное значение сохраняется.
- distance_reduction_pct = 100 × (baseline_distance - candidate_distance) / baseline_distance; при baseline = 0 -> null.

Упаковка может остаться узким местом: путь сократился, а число выполненных заказов не выросло. Интерфейс должен уметь показать такой результат без искусственного «улучшения» очереди. При улучшении отбора очередь упаковки способна даже вырасти.

Экономика P1: net_labor_saved_s / 3600 × labor_cost_per_hour. Ставка - введённое пользователем предположение в KZT, не факт о рынке. Расчёт труда не равен автоматическому снижению фонда оплаты труда или прибыли.

11. Архитектура и структура будущей кодбазы

Поток: raw XLSX > очистка/агрегаты > временные признаки > прогноз > ограниченный план > graph routing + discrete-event simulation > run result > 3D replay + KPI.

Предлагаемые будущие директории:

```
frontend/
  src/app/            экран, панели, состояние запроса
  src/scene/          GLB, камера, выбор ячеек, два viewport
  src/replay/         clock, seek, интерполяция, trails
  src/contracts/      TS-типы по согласованному JSON
  public/models/      warehouse.glb, picker.glb, crate.glb
```

```
backend/
  app/api/              health, catalog, forecast, plans, compare
  app/data/             загрузка, фильтры, срез, признаки
  app/forecast/         baseline, ML, temporal backtest
  app/planning/         ABC, swap search, relocation execution
  app/simulation/       graph, routing, queues, events, metrics
  tests/                инварианты и несколько ручных сценариев
shared/                 layout, defaults, contract examples
artifacts/              версии данных, прогнозов и demo runs
docs/                   допущения, лицензии, запуск, результаты
```

API P0:

- GET /api/health > version, status.
- GET /api/catalog > layout, SKU, доступные test dates, параметры и provenance.
- POST /api/forecast > cutoff/date, модель, SKU predictions, baseline predictions, validation summary.
- POST /api/plans > forecast_id + baseline_assignment_id + allowed_swaps 0/1/2 > plan list. Сервер проверяет совместимость версий.
- POST /api/compare > scenario_id + baseline_assignment_id + candidate_plan_id + config > парный результат. Клиент не отправляет произвольные пути или готовые KPI.

Ответы immutable и имеют schema_version, layout_version, data_version/config_hash, scenario_id, forecast_id/plan_id, source hashes. Нет требования к полноценной БД: ID может быть стабильным хешем нормализованного содержимого. Повторный расчёт с теми же входами даёт тот же содержательный результат; случайный request_id/timestamp не участвует в проверке детерминизма.

Валидация: неизвестный SKU, дублирующее назначение ячейки, отрицательная quantity, отсутствующее ребро, несовместимые версии, пустая история, недостижимая упаковка возвращают понятный 422 с кодом и полем. 500 не должен маскировать неверные входы. HTTP/CORS ограничить localhost в локальном режиме; .env.example не содержит секретов.

Основной рабочий режим - API. Резерв на защиту - заранее рассчитанные JSON реальных прогонов той же версии ядра, встроенные в frontend. В резервном режиме есть надпись «сохранённый расчёт»; параметры, требующие пересчёта, заблокированы. Нельзя делать вид, что отключённый backend выполнил новое моделирование.

12. Требования к 3D и производительности

Поставки моделлера: исходный .blend, runtime GLB, manifest объектов и preview с размерами. Все SKU - логические данные и цветные этикетки; не моделировать двадцать разных подарочных товаров. Создать один качественный стеллаж и коробку, затем повторять. Два работника различаются одеждой/ID; сложный motion capture не нужен.

Имена S_A1_B1…S_A4_B5 соответствуют slot_id, а SKU привязывается через assignment. У mesh нет навсегда закреплённого артикула. Объект PACK и буфер имеют отдельные имена. Масштаб экспортируется 1.0, корни не повернуты неожиданно, пол Y=0. Настройка Blender Z-up преобразуется экспортёром glTF в runtime Y-up; не вращать потом всю сцену второй раз.

Целевые бюджеты собственного склада: GLB ≤ 5 MB, желательно ≤ 100k видимых треугольников на один склад, ≤ 150 draw calls на парный показ в low quality, материалы совместно используются. Это инженерные цели, не уже измеренная скорость. Донор имеет много отдельных meshes, его целиком копировать в два viewport без оптимизации рискованно.

Один WebGLRenderer с двумя viewport/scissor либо один общий облегчённый план при слабом устройстве; pixelRatio в low mode ≤ 1.25. В low mode отключить прозрачные стены, постэффекты и динамические тени. Нужен

ясный интерьер и читаемые товары, а не фасад промышленного парка.

Проверка на целевом ноутбуке: зафиксировать модель устройства/браузер, viewport 1366×768, режим качества, median FPS и p95 frame time за 60 секунд парного проигрывания. Ориентир ≥ 30 FPS; если не проходит, включить low mode и уменьшить геометрию. Нельзя обещать FPS без фактического замера.

13. Приёмка

Функциональные и численные критерии вынесены в 05_Acceptance_and_Demo.md. Минимальный gate: реальные заказы проходят полный цикл; два прогона различаются только размещением; ни один маршрут не проходит сквозь стеллаж; счётчики воспроизводятся после seek/reset; переносы оплачены; отрицательный эффект не скрывается; одна команда запуска работает без внешних API.

Приоритет интеграции: сначала серые кубы + корректные реальные события, затем модели и оформление. Финальный проект включает собственные прогноз/перестановки/маршруты/события/метрики; донорский виджет с новыми подписями не удовлетворяет ТЗ.

14. Что показывать на защите

Короткая последовательность: текущий склад > прогноз на дату > две пары товаров подсвечены > цена перестановки > синхронный поток заказов > разница пути/труда/очереди > честный итог по нескольким датам. Показывать факт наличия датасета и backtest, не использовать ненаблюдавшиеся «достигнутые 30%».

Название «склад перестраивается» означает перемещение товаров между существующими адресами. Стеллажи в MVP не двигаются, план здания не меняется. Главная ценность - проверка ограниченного решения до смены, включая ситуацию, когда ничего менять не стоит.

15. Первичные источники

- UCI Online Retail II и лицензия.
- scikit-learn: временные лаги и временная оценка.
- SimPy: ресурсы и очереди.
- SciPy assignment solver: полезен для назначения без дополнительных ограничений; бюджет swap требует отдельной логики.
- Google OR-Tools assignment: возможное развитие, не обязательная зависимость MVP.
- Проверенный визуальный донор.
- Lucas Systems: существующее коммерческое slotting-решение.

Алгоритмы, геометрия, параметры и распределение работ в этом пакете - проектные решения команды, а не подтверждённые свойства реального склада.

Два участника: зоны ответственности и 48 часов

Оценка: примерно 25 часов предметной работы на человека плюс 3 часа совместной интеграции/проверки. Это ориентир для двух человек, работающих с AI-помощниками, а не обещание срока. В 48 календарных часов остаётся время на сон и резерв. При меньшем доступном времени действуют сокращения ниже.

A: 3D, клиент, воспроизведение

Владелец frontend/src/scene, frontend/src/replay, frontend/src/app, frontend/public/models, исходников Blender и визуальной документации. Основной профиль - моделирование и 3D, но роль включает связывание сцены с данными, интерфейс и визуальную проверку. Если человек умеет только моделировать и вообще не готов править TS даже с AI, это не равное разделение: тогда придётся сократить интерфейс до одного вида и заранее рассчитанного JSON.

Задача	Часы	Конкретная поставка	Приёмка
A1. Запуск донора, удалить нерелевантные панели/AGV-циклы	2	Минимальный Vue/Three экран, камера, loader	Работает production build, нет декоративных KPI
A2. Graybox и калибровка метра/осей	2	Пол, 20 секций, PACK/BUFFER, debug graph	Проверочный маршрут виден и не режет стеллажи
A3. Blender-модули и GLB	4	Rack bay, crate, low-poly picker/cart, pack station, исходный .blend	Единицы/IDs/anchors совпадают с manifest
A4. Сцена и связывание адресов с SKU	3	Загрузка/инстансы, label, выбор SKU/slot, plan highlights	SKU меняет место, slot сохраняет адрес
A5. Replay по фикстуре и реальным событиям	5	Clock, walk/pick/dropoff, pause, reset, seek, trail	Точное положение на заданных временах, нет двойных событий
A6. Интерфейс сравнения и планов	4	Два viewport либо резервный toggle, KPI/таблица, запросы API	Общие clock/camera; цифры из результата B
A7. Производительность и ошибки	3	Low mode, состояния loading/error, тест на ноутбуке	Читаемо 1366×768; зависание/ошибка не дают пустой экран
A8. Демо и передача	2	Короткая запись, 3 скриншота, инструкция запуска клиента	Демо повторяется из чистого состояния
Итого	25		

A не должен обучать модель, рассчитывать оптимальный путь, определять длительность отбора или изобретать KPI. Он может работать по приложенной искусственной фикстуре до появления backend.

B: данные, модель, планирование, симуляция, API

Владелец backend, подготовки artifacts, численных тестов и backend-документации. Канонический JSON контракт и shared layout меняются только после согласования с A. Готовые агрегаты и CPU-probe в пакете уменьшают начальную работу, но не заменяют интеграцию.

Задача	Часы	Конкретная поставка	Приёмка
B1. Данные и reproducibility	2	Подключить готовые CSV, проверить manifest и фильтры	Cutoff, SKU selection, hashes и coverage воспроизводятся

Задача	Часы	Конкретная поставка	Приёмка
B2. Граф и маршруты	3	31/33 graph, Dijkstra, nearest-neighbor, route export	Ручные 8/21.2 м проверки, путь только по рёбрам
B3. Два комплектовщика и упаковка	5	SimPy процессы, FIFO, состояния заказов, интервалы	Полный цикл и ручной однозаказный результат
B4. Прогнозы и backtest	3	Фиксированный CPU-ML + lag7/mean28, 20 дат	Нет future leakage; результаты не подменены лучшим днём
B5. ABC и до двух обменов	4	0/1/2-swap планы, transfers, стоимость	Буфер пуст в конце, ≤4 SKU, нет выгодного плана -> no-op
B6. Парное сравнение/API/артефакты	3	catalog/compare минимум, общий scenario hash, JSON	Входные заказы одинаковы, клиент видит реальные события
B7. Проверки и массовый прогон	3	20-date result CSV, sanity cases, README	Сохранены положительные/нулевые/отрицательные итоги
B8. Офлайн-набор и объяснение результата	2	Demo JSON, assumptions, таблица фактов	Запускается без LLM/внешнего API, числа совпадают
Итого	25		

В не должен параллельно строить красивый frontend, делать скелетную анимацию или переписывать донор в другой стек. Он отдаёт небольшие законченные пакеты данных, а не огромный незавершённый backend в последний час.

Совместные контрольные точки

Календарный срок от начала	A	B	Общий результат
H0-H1	Проверка донора/Blender	Проверка данных/окружения	Заморозить координаты, ID, единицы, контракт, стек
H1-H4	Graybox + loader + упаковка	Graph + ручной маршрут + каркас run result	Первое движение на искусственной фикстуре
H4-H8	Повторяемые модели и ID	Два picker + pack queue	Реальный завершённый заказ на серых кубах
H8-H12	Pause/reset и события picking	Экспорт событий и KPI, первые API	Один законченный сквозной сценарий, пусть некрасивый
H12-H18	SKU labels, trails, панели	Прогноз, ABC, swap-cost	План меняет размещение; метрики пересчитаны
H18-H24	Парный вид/общая камера	Все методы на одинаковом потоке	Первый честный A/B и проверка чисел
H24	Оценить оставшийся фронт	Оценить оставшуюся математику	Применить cut line, заморозить новые функции
H24-H34	UI/low mode/seek QA	20-date backtest/edge cases	Стабильная сборка и таблица всех результатов
H34-H40	Демо/скриншоты	Офлайн JSON/assumptions	Чистый локальный запуск на ноутбуке защиты
H40-H44	Повторить выступление	Проверить цифры и ответы	Две успешные репетиции без интернета
H44-H48	Резерв	Резерв	Только исправления критических дефектов

Таблица задаёт доступность поставок, а не непрерывную работу без сна. Окна отдыха согласовать после первого сквозного прогона; второй человек должен иметь сохранённую фикстуру и инструкции.

Правила параллельной работы

1. A и B не редактируют чужую директорию без сообщения. Договорённости о shared/layout и shared/contracts фиксируются коротким changelog, не устным «я там чуть масштаб поменял».
2. H1: A получает приложенные layout + hand-check run. H8-H12: B заменяет фикстуру реальным run той же schema_version. A меняет URL источника, а не переписывает проигрыватель.
3. Товар хранится в assignment, а адрес в layout. Изменение оформления GLB не меняет slot_id или пути.
4. В первые 12 часов интегрироваться каждые 3-4 часа по одному законченному заказу. Не ждать готовности всех моделей/всех метрик.
5. В PR/commit описывать конечное поведение и проверку. Не переносить целиком чужой WMS ради готового названия функции.
6. Несовпадение версий или координат останавливает новый прогон. Не «чинить» scale на стороне клиента без изменения manifest.

Cut line: что сокращать, если не укладываемся

Порядок сокращения:

1. LLM объяснения, деньги, импорт пользовательских CSV и произвольные настройки.
2. Текстуры, стены/крыша, декоративный транспорт, сложная походка, постэффекты.
3. Два viewport заменить одним с A/B toggle и общей сохранённой шкалой времени. Итоговая таблица обоих вариантов остаётся рядом.
4. Раздельные forecast/plans endpoints заменить готовым catalog и одним compare; фактические расчёты и версии сохраняются.
5. Произвольную перемотку заменить надёжными reset/pause и несколькими точками времени, если полный seek не успевает пройти проверку.
6. Оставить одну основную дату для живого опыта; остальные 20 дат показать из реально выполненного offline benchmark. Не изображать сохранённые результаты новым онлайн-расчётом.

Нельзя сокращать: одинаковые заказы для A/B, маршрут по проходам, возврат/упаковку, стоимость переносов, временное разделение данных, простой baseline, честные нулевые/отрицательные результаты и маркировку модельных допущений. Именно эти свойства отличают рабочий прототип от заранее нарисованной анимации.

Готовые задания AI-помощникам

Для A: «Реализуй только visual client по 01, 03 и contracts. Владей frontend и Blender assets. Сначала воспроизведи hand_checked_run.json, затем подключи API. Не вычисляй собственные KPI/пути, не меняй IDs и не редактируй backend. Другой разработчик работает параллельно; сохраняй его изменения».

Для B: «Реализуй только backend по 01, 05 и contracts. Используй подготовленные данные, фиксированный ABC и 20-date temporal evaluation. Сначала graph + one-order simulation + exporter, затем прогноз и swaps. Владей backend/artifacts; не меняй frontend/GLB/контракт без согласования. Другой разработчик работает параллельно; сохраняй его изменения».

Проверка и демонстрация Qoyma Twin

Численные проверки до красивой сцены

Проверять предметные инварианты, а не создавать десятки тестов, копирующих реализацию. Допуск расчётных времён/расстояний 1e-6; округлять только UI.

ID	Вход	Ожидаемый результат
G1	Канонический layout	31 узел, 33 неориентированных ребра, все слоты достижимы от PACK
G2	(3,0,4) > (7,0,4)	8 м через передний проход; прямая 4 м запрещена
G3	PACK > A1B1 > A2B1 > PACK	8.6 + 8 + 4.6 = 21.2 м
S1	Один заказ, qty1 на A1B1 и A2B1, arrival0	P1: 21.2/1.2 с движения + 2×6.15 с отбора +3 с сдачи; ready32.9666667, pack24.1, completed57.0666667
S2	Два одновременных заказа	P1/P2 получают по одному; один упаковщик не обслуживает два заказа одновременно
S3	Пустой поток	Нулевые суммы/очередь, null для неопределённых ratios/percentiles, приложение не падает
S4	Высокая стоимость упаковки	Может вырасти очередь, выгода пути не обязана увеличить completed_by_horizon
P1	План без изменений	Те же assignment и заказы -> те же события/метрики, relocation=0
P2	Один swap занятых слотов	Три transfers, шесть take/place, буфер свободен в конце, два SKU перемещены
P3	Два swap	Не более четырёх разных SKU; ни один SKU не участвует в двух swap
P4	Выгода меньше стоимости переносов	No-op выбран как лучший; отрицательный net effect не маскируется
M1	Повторный прогон тех же входов	Одинаковые содержательные JSON и KPI; wall time/request_id исключены
M2	Invoice с qty100 одного SKU	Одно посещение, qty100 в handling; не100 походов
M3	Отсутствие спроса	Нулевой прогноз допустим, WAPE=null при sum(y)=0

Проверка справедливости и ML

- SKU отобраны до cutoff, test dates зафиксированы до просмотра эффекта.
- Lags и rolling features используют только прошлое, включая уже раскрытые предыдущие дни test. Random train/test split не используется.
- Baseline ABC построен из 28 дней до 2011-10-03 и одинаков для каждого независимого дневного опыта.
- Для mean28/lag7/ML одинаковы swap budget, costs, routing policy, geometry, staffing и arrivals.
- Массив входных заказов и длительности операций материализованы до запуска A/B. Если позже добавится шум, значения сохраняются по order/line ID заранее; один и тот же seed внутри двух разных event loops

недостаточен.

- Будущие корзины используются только для проверки, не для выбора плана. Если UI уже раскрыл факт, повторное построение плана всё равно использует прежний forecast snapshot.
- В таблице по 20 датам есть все дни, а не только выигрышные. Показать число дней с положительной чистой экономией и суммы/медиану, не только лучший процент.
- При агрегации WAPE складывать абсолютные ошибки и факты; не усреднять дневные проценты с разными знаменателями. Дневные p95 нельзя называть общим p95: его считать из объединённых наблюдений.

Проверка 3D

1. Линейка 1 м и axis markers совпали после GLB-экспорта. Расхождение bin/pick anchors с manifest ≤ 0.02 м.
2. На G2 аватар идёт через front aisle. Polyline не сглажена сквозь rack.
3. Между концом WALK и следующим WALK фигура стоит на месте во время PICK/DROPOFF.
4. На 7.1666667 с P1 у A1B1; на 13.3166667 с он закончил первый pick; на 32.9666667 с корзина доступна упаковке.
5. Pause останавливает clock; изменение playback speed не меняет итоговые KPI.
6. Seek назад, затем вперёд не удваивает заказ/количество. Reset убирает маршруты, очередь и счётчики.
7. Выбор SKU в двух вариантах показывает один товар в разных slot, если план его переместил.
8. Выключенный backend оставляет понятный error или явно помеченный сохранённый run. Произвольные параметры не применяются к старому результату.
9. Проверить low mode на ноутбуке защиты: читаемость/камеру/FPS, а не на скриншоте мощного компьютера.

Чек-лист готовности

- Production build frontend проходит; backend запускается по README из чистого терминала.
- Все runtime assets и demo runs локальные. Нет зависимости от CDN, платного API или доступности GitHub.
- THIRD_PARTY_NOTICES содержит лицензии реально использованного кода; источник и обработка UCI подписаны.
- Manifest и результат показывают одинаковые schema/layout/data versions.
- CSV benchmark и экран показывают одинаковые итоговые цифры.
- README отделяет «измерено на данных» от «задано в модели» и «ещё не реализовано».
- Нет фразы о гарантированной прибыли/росте выпуска или реальном клиентском внедрении без подтверждения.

Демонстрация на 60 секунд

0-8 с: «Мы проверяем, какие товары стоит поменять местами до смены. Вот существующее ABC-размещение».

8-18 с: показать дату, прогноз и вариант до двух обменов; выделить SKU и затраты на перенос.

18-42 с: запустить два одинаковых потока, ускорить clock; показать маршрут и очередь упаковки. Избегать длинного ожидания загрузки, загрузить модель до начала.

42-53 с: итог по выбранному дню: метры, чистый труд, время заказа. Если выпуск не вырос, назвать упаковку узким местом.

53-60 с: таблица всех тестовых дат и одно предложение о том, что продажи реальные, а склад/операционные времена смоделированы.

Если на первой тестовой дате выгодного плана нет, не менять её тайно на лучшую. Можно продемонстрировать заранее описанный синтетический сдвиг спроса как отдельный сценарий, а исторический результат оставить рядом. У такого сценария отдельный ID, provenance и подпись.

Вопросы жюри и содержательные ответы

«Где здесь AI?» - обученный прогноз частоты отбора; его качество сравнивается с простыми моделями. Планирование и очередь - отдельные расчётные методы. LLM не выдаётся за оптимизатор.

«Почему это двойник?» - цифровое представление одной геометрии и операционной модели, в которой 3D и KPI используют одни события. Сейчас это прототип; live-синхронизации с физическим складом нет.

«Почему не просто положить популярное ближе?» - это наш исходный ABC baseline. Проверяем добавочную пользу ограниченного изменения под завтрашний прогноз и оплачиваем труд переноса.

«Меньше ходьбы - значит больше заказов?» - не обязательно: упаковка может ограничивать выпуск. В модели эти показатели разделены.

«Данные Казахстана?» - нет, открытый британский набор. Следующий шаг - местный пилот с экспортом WMS и замером операций; результат UCI не переносится автоматически.

«Чем отличаетесь от коммерческих аналогов?» - существование slotting-решений признаём; сейчас защищаем работающий прозрачный workflow небольшого склада, проверенный код и воспроизводимый эксперимент. Цена и коммерческий спрос пока гипотезы.

«Что нужно от пилота?» - план проходов, SKU > slot, история строк заказов минимум несколько недель, скорость/время операций и ограничения вместимости. Затем replay реальной смены и проверка калибровки до рекомендации переносов.